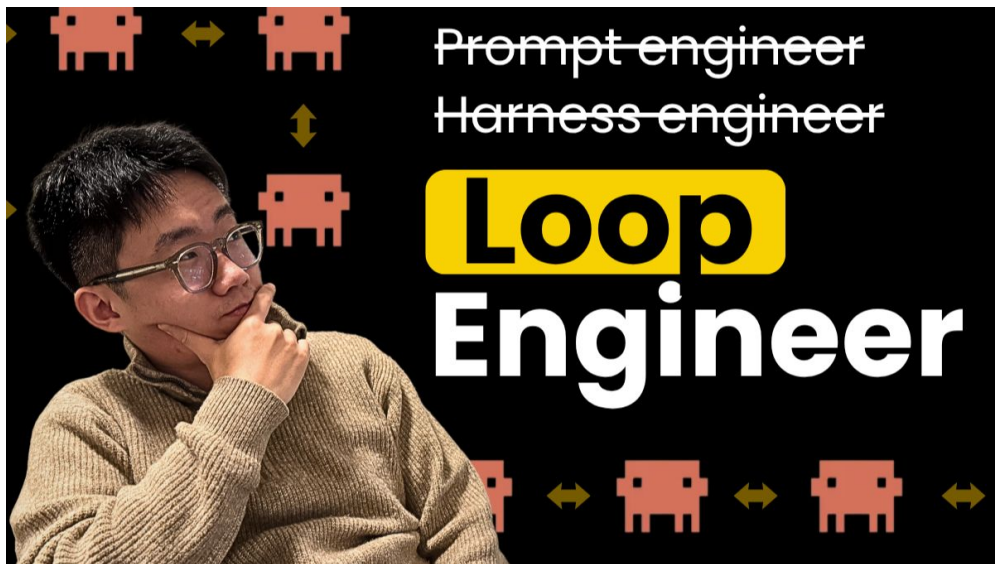


课程笔记

从 Prompt Engineering 到 Loop Engineering: AI Agent 的下一层用法

SSS & Codex

2026 年 6 月 22 日



视频作者/频道: AIJasonZ

发布日期: 2026 年 6 月 19 日

视频时长: 20:24

视频链接: <https://www.bilibili.com/video/BV1tSje6REDD/>

目录

1 引子：从手写 prompt 到自动运行的 loop	3
1.1 开场案例：PR、goal loop 与页面产出	3
1.2 从手写 prompt 到自动发起工作的系统	3
1.3 本章小结	4
2 概念阶梯：Prompt Engineering、Context Engineering、Harness、Loop Engineering	4
2.1 第一层：Prompt Engineering 解决一次调用如何被引导	4
2.2 第二层：Context Engineering 解决相关信息如何进入窗口	5
2.3 第三层：工具调用、skills 与跨会话工作	6
2.4 第四层：agent harness 把模型放进可运行系统	6
2.5 本章小结	7
3 Loop Engineering 的核心机制：触发、调查、分配、学习	8
3.1 Trigger：让 agent 在合适时刻醒来	8
3.2 Investigate & Act：先调查事实，再采取行动	8
3.3 Backlog gen / Assign task：把发现转成下一步工作	9
3.4 Review & learn：把本轮经验写回系统	9
3.5 Support loop：从回复工单到触发 PR	10
3.6 本章小结	10
4 复利系统：support、SEO、ads、product growth 共享同一个 brain	11
4.1 Signal：把重复摩擦变成持久文件	11
4.2 多个 loop 共享同一个业务大脑	12
4.3 复利来自共享格式，而不是更多 agent	12
4.4 四个核心组件：trigger、file structure、tools、verify	13
4.5 本章小结	14
5 Codebase Harness：让 agent 看得懂、跑得起、验得准	14
5.1 可读代码库：把上下文工程写进仓库	14
5.2 可执行环境：让本地启动成为一条命令	16
5.3 可验证性：证据优先于自我感觉	17
5.4 从代码库设置到 Loop Engineering	18
5.5 本章小结	18
6 文件、契约与日志：把 loop 变成可维护系统	18
6.1 Artifact：agent 工作留下的结构化产物	18
6.2 Signal：多个 loop 共享的业务神经末梢	20
6.3 Loop contract：每个 domain loop 的 README	21
6.4 Global work log：组织记忆与临时上下文	21
6.5 本章小结	22

7 落地示例：搭建第一个 support loop	22
7.1 先定义 loop 要处理的真实工作	22
7.2 准备 skills 和 connectors	23
7.3 写入业务背景文件和工程规则	24
7.4 用 architecture markdown 生成 artifact 和 domain 框架	24
7.5 先手动测试，再设置 trigger	25
7.6 把模板仓库当成可复制的起点	27
7.7 本章小结	27
8 总结与延伸：Loop Engineering 的工程判断	27
8.1 讲者的实质性收尾	28
8.2 一个压缩系统模型	28
8.3 实践落地顺序	29
8.4 风险清单	30
8.5 读者应带走的能力	30

1 引子：从手写 prompt 到自动运行的 loop

凌晨一点，代码库里连续出现 PR；讲者给出的解释是，各种不同的 agent loop 正在自动发现问题并接手工作。00:00:00--00:00:11 这个开场直接建立本片的核心问题：当 coding agent 已经可以持续运行、持续提交、持续产出业务结果时，人的主要工作会从逐条撰写 prompt，升级为设计让 agent 能反复做对事的运行系统。

1.1 开场案例：PR、goal loop 与页面产出

视频的第一个证据很具体：凌晨约一点，代码库中有大量 PR 被持续提交；这些提交来自不同智能体循环自动发现问题、接手任务。00:00:00--00:00:11 第二个证据更能说明 loop 的商业含义：讲者提到一个 goal loop 已经连续运行两天，每天产出 20--40 个高质量页面，并在他完全无需过问的情况下给公司带来流量。00:00:11--00:00:21

什么是 goal loop

goal loop 可以理解为围绕一个长期目标持续运行的 agent loop。一次普通 prompt 像给实习生发一条临时指令；goal loop 更像给一个小团队设定目标、资料夹、检查规则和交付节奏。它会在多轮执行中保留状态，继续推进尚未完成的工作，并把产出落到代码、页面、任务或日志里。

这个案例的动机很直接：如果 agent 只能在一次会话里回答问题，prompt 写得再漂亮，价值仍然受人的注意力限制；如果 agent 能在合适的触发条件下醒来、调查、行动、留下结果，并让下一轮继续使用这些结果，系统就开始拥有持续生产能力。视频标题里的 Loop Engineering 正是沿着这个问题展开。

1.2 从手写 prompt 到自动发起工作的系统

讲者随后把主题压缩成一句工程判断：上周大家讨论的重点，是设计能够自动给 coding agent 发起工作的 loop。00:00:27--00:00:34 这里的关键词是系统级的工作选择：谁决定下一步工作。手

写 prompt 的模式里，工作选择、上下文整理、结果验收、后续跟进都集中在人身上；loop 模式把这些环节拆成触发器、状态文件、日志、artifact、工具和验证证据，使 agent 在被唤醒后能知道自己该调查什么、该产出什么、该把信号留在哪里。

Loop Engineering 的人口定义

Loop Engineering 是为 agent 设计可持续运行环境的工程方法。它关注的核心问题是：系统在每一轮如何被触发，如何携带上下文，如何选择有价值的任务，如何产出可复用 artifact，如何记录 signal 与 log，如何通过 verification 证明结果可靠。

可以用一个很朴素的类比来理解：prompt 像一次性点餐，loop 像一条带库存、排班、质检和复盘的厨房流水线。一次点餐能得到一道菜；流水线能每天稳定出餐，还能根据顾客反馈调整菜单。Agent 领域里的差别也在这里：单次 prompt 解决眼前任务，loop 让任务选择、执行、记录和改进变成可重复的系统行为。

手写 prompt 的隐藏瓶颈

当每一轮工作都依赖人来回忆背景、复制资料、粘贴 prompt、检查结果，系统吞吐量会被人的注意力和记忆力卡住。粗糙地把同一条 prompt 反复投喂给 agent 也会遇到同类问题：缺少明确状态、缺少证据、缺少学习路径，循环会把错误和重复劳动一起放大。

1.3 本章小结

本章用视频开场的两个事实建立问题意识：其一，代码库能在夜间持续收到 agent loop 生成的 PR；其二，一个 goal loop 能连续两天每天产出 20--40 个高质量页面并带来流量。由此可见，本片讨论的重点是如何把 agent 从“等待人工 prompt 的执行人”组织成“在工程环境中持续行动的工作单元”。下一章将把这个变化放进更长的技术阶梯里：从 Prompt Engineering，到 Context Engineering，再到 agent harness 与 Loop Engineering。

2 概念阶梯：Prompt Engineering、Context Engineering、Harness、Loop Engineering

开场案例提出了问题，第二章负责把术语放回技术演进里。讲者提醒，过去一年出现了许多新词，这些词看起来混乱，其实分别对应大语言模型应用的不同层级。00:00:50--00:01:00 这一节的目标，是把四个层级讲清楚：Prompt Engineering 处理一次调用的指令质量，Context Engineering 处理有限窗口里的信息选择，agent harness 处理模型之外的运行支撑，Loop Engineering 处理整个 agent 系统下一步该做什么。

2.1 第一层：Prompt Engineering 解决一次调用如何被引导

讲者把时间拉回 2023 年：GPT-3.5 或 GPT-4 API 刚出现时，大多数任务都很简单，主要是文本补全；使用者给 API 一个输入，然后让模型输出文本，例如抽取结构化数据或写博客。00:01:00--00:01:15 由于模型输出带有不确定性，Prompt Engineering 这个术语开始流

¹ 视频画面时间区间：00:00:50-00:01:00。

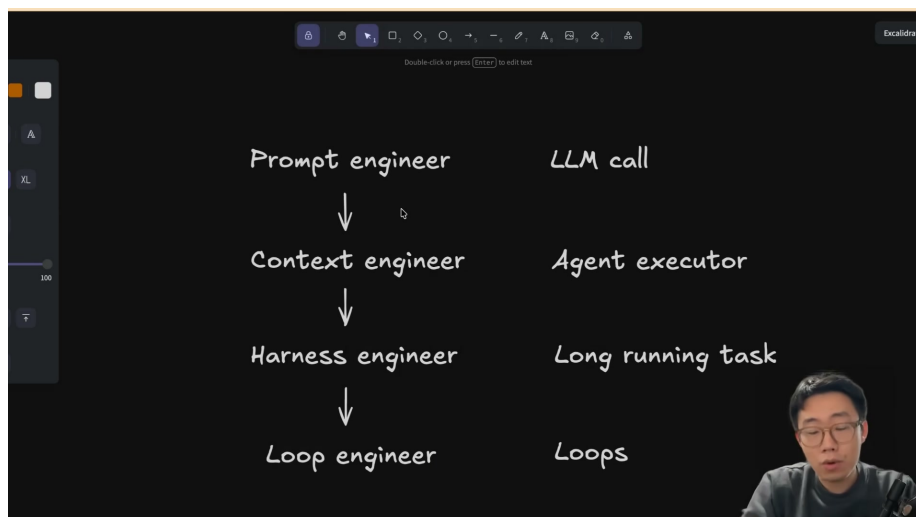


图 1: 从 Prompt engineer 到 Loop engineer: 任务跨度越长, 模型外部系统越重要。¹

行: 它研究如何在调用大语言模型时注入合适的指令、格式、风格和约束, 从而引导模型行为。

00:01:15--00:01:26

Prompt Engineering 的定义

Prompt Engineering 是围绕单次 LLM 调用设计输入的技术。它通过任务说明、输出格式、风格约束、示例和边界条件, 提升一次生成的可控性。它适合文本补全、信息抽取、文章生成等短任务, 是后续 agent 能力的底层组成部分。

一个直观例子是视频里提到的“始终以全大写形式返回文本”, 模型就会按指定风格写作。

00:01:26--00:01:32 这说明 prompt 可以塑造局部行为。可是当任务开始跨工具、跨文件、跨多轮会话推进时, 问题会扩展到更大的范围: 模型需要知道哪些背景最相关, 哪些旧信息已经过时, 哪些工具结果需要继续保留。

2.2 第二层: Context Engineering 解决相关信息如何进入窗口

到 2024 年中期, 模型能力增强, 上下文窗口也显著扩大。讲者给出了一组数字: 早期窗口约 4k token; 128k token 出现时已经很惊人; 随后 Google 把 1M token 上下文推到默认配置级别。

00:01:32--00:01:52 窗口变大改变了使用方式: 系统可以把工具调用、工具响应和更多任务背景放入对话, 使模型持续推进, 直到它判断任务完成。00:01:52--00:02:10

token window 与有效窗口

上下文窗口是模型一次能读取的文本容量; 有效窗口强调其中真正可用、相关、可被模型稳定利用的信息区域。讲者指出, 虽然有 1M token 的窗口, 实际有效窗口常落在 128k--200k 之间。00:02:17--00:02:23 这像仓库面积与拣货效率的关系: 仓库越大, 可以放的东西越多; 货架混乱时, 拣货员仍然会找不到关键零件。

²视频画面时间区间: 00:02:17-00:02:28。

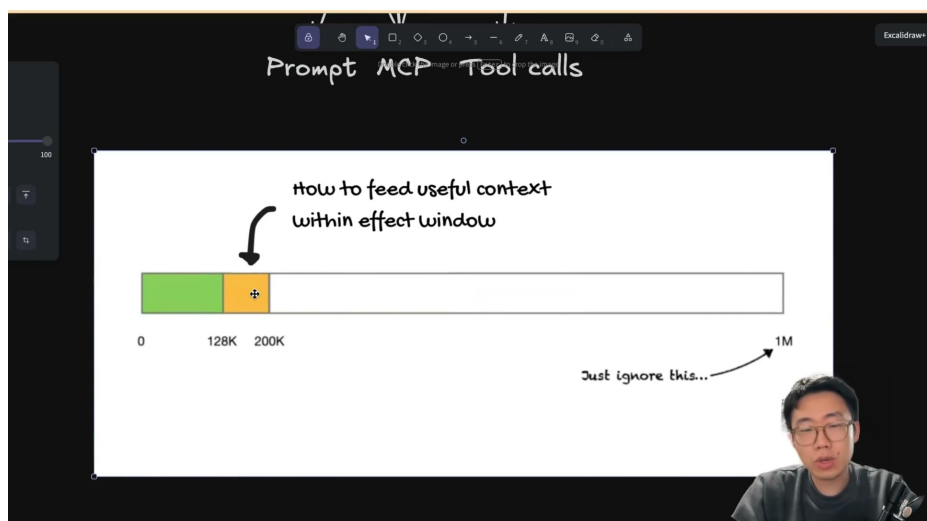


图 2: 名义上下文窗口可以很大，真正影响结果的是有效窗口内的高相关信息。²

大窗口的常见误判

大上下文容量会被长任务快速消耗。真正影响 agent 表现的，是能否在窗口内提供最相关的信息。00:02:23--00:02:28 把所有材料一股脑塞进去，会增加噪声、冲淡关键约束，并让后续工具结果挤压前面的任务状态。

因此，Context Engineering 的重点是选择、压缩、缓存、注入。视频列举了系统提示词中的缓存触发、长对话压缩策略，以及 skills 这类新概念：skill 能把特定任务流程和工具知识封装起来，扩展 agent 能力，同时减少对上下文窗口的占用。00:02:28--00:02:46

2.3 第三层：工具调用、skills 与跨会话工作

当模型能使用工具，agent 的行为就从“生成一段文本”扩展为“调查环境、调用工具、读取结果、继续行动”。视频在 00:01:56--00:02:08 提到 MCP、tool calls 和响应会被包含在对话中，让模型在循环里持续推进。这里的关键变化是反馈链条：工具结果进入上下文，模型据此选择下一步，下一步又可能产生新的工具结果。

到 2025 年底，任务长度进一步上升。讲者说，人们开始让模型处理更长、更庞大的任务；从当年年初起，通过一次性调用 Claude Code 完成 30 分钟甚至两小时的工作量变得普遍。00:02:46--00:03:01 随后出现了粗糙的 while loop、goal、loop、workflow 等概念。00:03:01--00:03:16 这些尝试共同指向跨会话工作：多个 agent session 可以各自处理一个任务，并在循环中运行到完成。00:03:18--00:03:32

粗糙 while loop 的缺口

简单 while loop 只是用同一条 prompt 反复运行 Claude Code。00:03:04--00:03:11 这种做法容易重复同一判断，缺少跨会话状态，缺少任务优先级，缺少验证证据，也缺少从上一轮失败中学习的机制。循环要产生复利，需要文件系统、日志、artifact、触发器和验证路径共同参与。

跨会话工作的直接后果，是系统需要跟踪不同会话之间的状态，或者提供某种共享文件系统，

让每个 agent session 都能理解进度并继续工作。00:03:33--00:03:43 这一步把问题推向 agent harness。

2.4 第四层：agent harness 把模型放进可运行系统

视频中这一处存在明显 ASR 错误，本文统一更正为 agent harness（运行马具）。讲者给出的定义很宽：harness 指模型以外的任何东西。00:03:49--00:03:58 这个定义之所以容易让人困惑，是因为它覆盖范围很大：状态文件、共享 filesystem、工具接口、编排逻辑、hook、日志、测试环境、验证证据，都可能属于 harness。

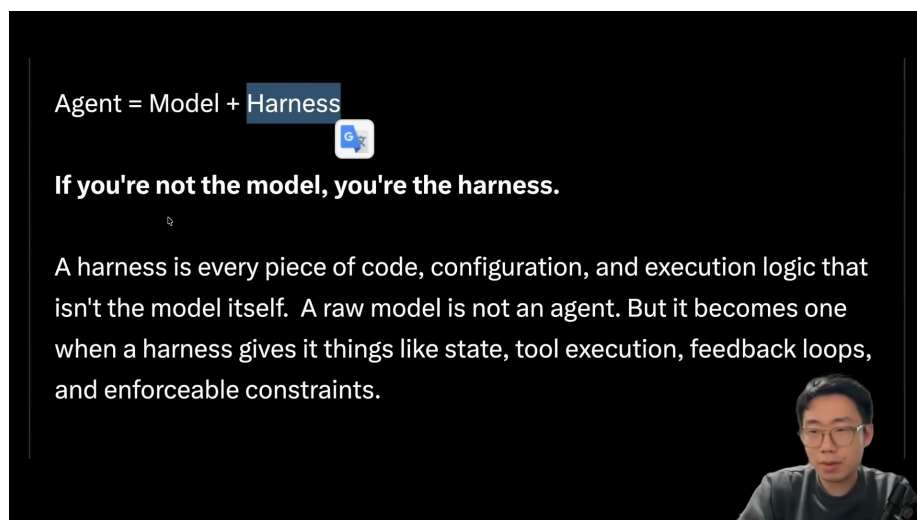


图 3: agent harness 把原始模型放进带状态、工具、反馈循环和约束的执行环境。³

agent harness 的定义

agent harness 是模型之外支撑 agent 工作的系统部分。模型像发动机；harness 像赛道、维修站、计时器、燃料补给和裁判系统。发动机决定瞬时动力，harness 决定它能否在真实赛程中稳定完成比赛。

讲者随后把优化问题分成两类。第一类关注 agent 循环本身，比如 Claude Code、Codex，或者自建 API agent 的执行质量：给它一个任务时，它能否很好地完成。00:04:10--00:04:26 第二类关注系统级工作选择：整个 agent 系统如何决定应该处理什么工作。00:04:26--00:04:35 后者就是视频里所说的 Loop Engineering。

Loop Engineering 的系统定义

Loop Engineering 是设计 agent 系统持续改进机制的工程方法。它要建立一个环境，使 agent 能被合适的事件触发，读取必要状态和日志，选择下一步有价值的工作，产出 artifact，并把结果反馈给后续循环。00:04:37--00:04:47

可以用一个压缩模型来记忆这四层关系：

single call → agent with tools → agent in harness → agent loops with system-level work selection

³视频画面时间区间：00:03:49–00:04:04。

前两层主要提升一次任务或一段会话的质量；后两层开始关注 agent 所处的工程环境，以及多个循环如何持续选择、执行、记录和改进。

2.5 本章小结

本章把视频前四分多钟的术语演进整理成四级阶梯。第一, Prompt Engineering 解决一次 LLM 调用如何被指令引导；第二, Context Engineering 解决大窗口时代的信息选择、压缩、缓存和 skill 封装；第三, MCP、tool calls、skills、goal、workflow 让 agent 进入长任务和跨会话工作；第四, agent harness 把模型放进包含文件系统、工具、日志、hook、编排和验证的运行系统里。由此引出的 Loop Engineering, 关注的是整个系统如何被触发、如何选择下一项工作、如何留下状态与证据。下一章会沿着这个定义进入 loop 的核心机制：trigger 如何唤醒 agent, agent 如何调查、行动、分配任务并在下一轮回顾学习。

3 Loop Engineering 的核心机制：触发、调查、分配、学习

上一章把 Loop Engineering 定义为系统级工作选择。本章进入它的最小运行模型：一个 loop 的目的并非反复投喂同一句 prompt；它要让 agent 在正确时刻被唤醒，读取当前状态，调查真实环境，采取行动，生成后续工作，再把本轮经验写回下一轮。视频在 00:04:47--00:05:30 给出的四步循环很紧凑：Trigger、Investigate & Act、Backlog gen / Assign task、Review & learn。

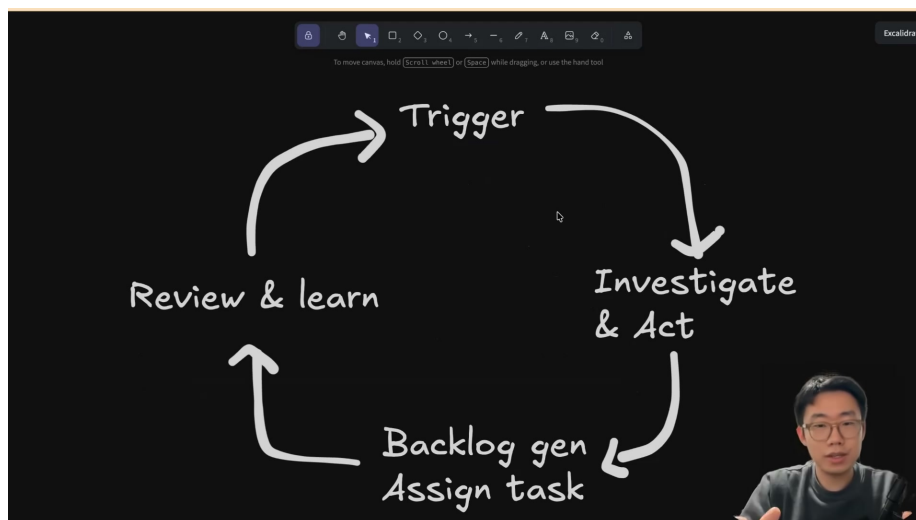


图 4: 一个可持续 loop 的基本节奏：触发、调查行动、生成待办或分配任务、回顾学习。⁴

3.1 Trigger：让 agent 在合适时刻醒来

讲者在 00:04:47--00:05:03 先列出触发来源：定时任务、另一个 agent、服务器故障时的 web-hook，都可以唤醒 agent，让它在无需人工参与的情况下交付有意义的工作。这里的 trigger 不只是“启动按钮”。它决定系统什么时候值得花计算、调用工具、访问业务系统和产出 artifact。

⁴视频画面时间区间：00:04:57--00:05:30。

Trigger 的定义

trigger 是唤醒 loop 的时间条件或事件条件。常见形式包括 cron、webhook、daemon、告警、用户行为、外部系统更新，以及另一个 agent 的交接信号。好的 trigger 应该让 agent 在“有新信息或新机会”的时候醒来，而不是无意义地空跑。

可以把 trigger 类比成医院分诊台。所有病人同时冲进手术室会造成混乱；分诊台根据症状和紧急程度安排处理顺序。Loop 系统也是这样：support loop 可以每 30 分钟查一次工单，SEO loop 可以每天上午 9 点抓取数据，故障处理 loop 可以由 webhook 立即触发。不同 trigger 体现不同业务节奏。

3.2 Investigate & Act：先调查事实，再采取行动

视频在 00:05:17--00:05:23 说，agent 每次通常会进行某种调查和行动，随后产生一系列待办事项或想法。这里的顺序很关键：先调查，再行动。调查意味着读取工单、日志、数据库、分析数据、文档和历史 artifact；行动可能是回复客户、起草 PR、创建 task、写 signal、更新文档或安排下一轮工作。

调查阶段的事实源

一个 loop 的质量很大程度取决于事实源。support loop 需要工单、支付、订阅、权限和后端日志；SEO loop 需要流量、关键词、页面表现和漏斗转化；product growth loop 需要产品分析、signal 和实验结果。事实源越稳定，agent 的行动越少依赖猜测。

这一步也解释了为什么 skills、MCP、connectors 和本地脚本属于 harness 的一部分。agent 如果只能读一段 prompt，它只能做语言层面的猜测；agent 能通过工具读取真实系统，才有机会把“建议”变成“行动”。在 Loop Engineering 中，工具承担把 loop 接到真实世界的管道角色。

3.3 Backlog gen / Assign task：把发现转成下一步工作

调查和行动之后，loop 需要把发现变成可接手的工作。视频在 00:05:21--00:05:30 提到，agent 会产生 todo 或想法，主 agent 确定优先级，并在需要时分配给其他 agent；下一次运行时再回顾和学习。这里的 backlog 可以是任务列表，也可以是 signal、工程 ticket、PR 计划、文档 TODO 或产品实验建议。

没有 backlog 的循环会失去记忆

如果每轮只输出一段总结，下一轮 agent 仍然需要重新判断“最重要的事是什么”。Backlog、signal 和 task 的作用，是把发现变成可排序、可分配、可验证的工作单元。缺少这些文件时，loop 很容易变成重复巡检。

这一点可以用队列模型直观理解。设一次 loop 产生的待处理工作为 B_t ，本轮完成的工作为 D_t ，新观察到的机会或问题为 N_t ，下一轮 backlog 可写成：

$$B_{t+1} = \text{prioritize}(B_t - D_t + N_t)$$

其中 prioritize 表示排序和筛选。公式的意思很朴素：loop 的下一轮不应从零开始，它应该继承未完成事项，扣除已交付工作，加入新发现，再重新排序。

3.4 Review & learn: 把本轮经验写回系统

循环的最后一步是回顾和学习。视频在 00:05:28--00:05:30 明确说, 这样下一次 agent 就可以进行回顾和学习。这里的学习指工程上的状态写回: 把结果、证据、阻碍、判断和下一步写进 artifact、signal、loop contract 或 log。

Learning 在 loop 中的工程含义

在这里, **learning** 指系统状态的更新: 把本轮调查结果、行动结果、失败原因、验证证据和下一步建议写回共享文件。它让下一轮 agent 的起点更高, 也让人类能复查 loop 为什么做出某个决定。

这和人类团队的复盘机制类似。一次值班结束后, 如果工程师只说“处理完了”, 下一位同事仍然不了解细节; 如果他更新 incident timeline、修复 task、监控链接和残留风险, 下一位同事就能直接接手。Loop 的 review & learn 承担同样角色。

3.5 Support loop: 从回复工单到触发 PR

讲者随后用支持工单举例。一个简单版本是: 每隔 30 分钟, agent 被定时任务唤醒, 查看所有支持工单, 回复尚未自动处理的工单, 并记录遇到的阻碍和想法, 供之后产品改进参考。00:05:31--00:05:49 这个 loop 本身已经有价值, 因为它把客户问题转成可积累的业务观察。

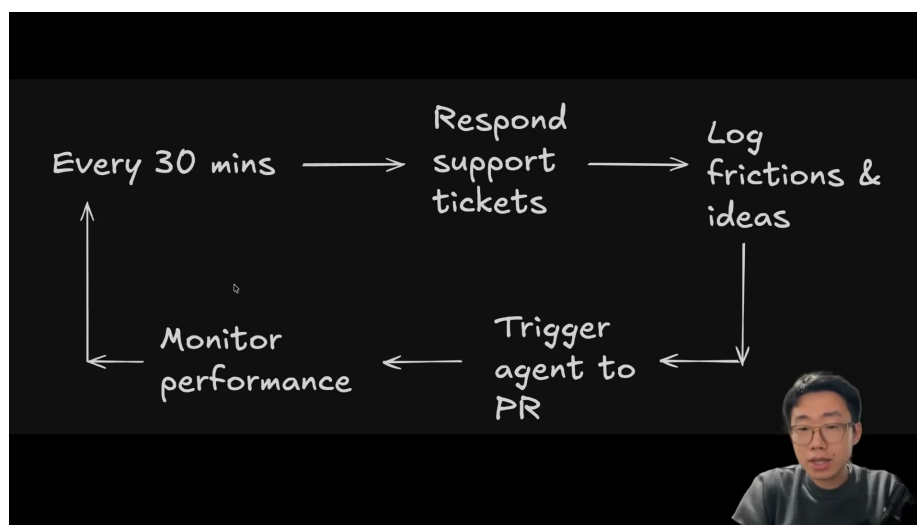


图 5: 支持循环可以从“处理工单”升级为“发现摩擦点、触发编程 agent、监控修复表现”的闭环。

5

更强的版本是: 如果 support loop 不只记录阻碍, 还能触发 coding agent 直接实现某些改进, 随后监控性能, 并告诉客户修复已经到位, 这个 loop 的价值会显著提高。00:05:53--00:06:06 这里出现了一个重要跃迁: support 不再只是响应客户问题, 它开始反哺产品和工程系统。

⁵ 视频画面时间区间: 00:05:31-00:06:10。

自动修复需要验证门槛

support loop 触发 coding agent 听起来很有吸引力，也更危险。客户反馈、工程改动、PR、部署和客户回复之间必须有明确边界。没有测试、只读 review agent、人工确认和回滚路径时，自动修复会把客服误判直接写进代码。

3.6 本章小结

本章把 Loop Engineering 的最小机制拆成四步：trigger 唤醒 agent，agent 调查事实并采取行动，系统把发现转成 backlog 或 task，最后通过 review & learn 把结果写回共享状态。Support loop 示例说明，一个每 30 分钟处理工单的简单循环已经能积累产品摩擦；当它进一步触发 coding agent、创建 PR、监控表现时，loop 就从客服自动化升级为跨支持、产品和工程的业务闭环。下一章会解释多个这样的 loop 如何通过共享 signal 文件夹形成复利效应。

4 复利系统：support、SEO、ads、product growth 共享同一个 brain

上一章展示了单个 support loop 的工作方式。本章讨论视频中更关键的系统效果：多个 loop 如果共享同一套 signal 和 artifact 文件，就会形成复利。讲者在 00:06:11--00:06:17 说，可以定义多个不同的 loop；如果日志系统设计得好，它们会产生复利效应。这里的“复利”不是金融隐喻的装饰，它指一个 loop 的发现会成为另一个 loop 的输入，系统越运行，业务观察越集中，任务优先级越清楚。

4.1 Signal：把重复摩擦变成持久文件

视频中第一个具体例子来自 support loop。它每 30 分钟处理知识工单，并把摩擦和想法记录到一个称为 `signals` 的文件夹中。00:06:21--00:06:30 讲者随后解释，signal 捕捉产品创意、发现的摩擦和可能错过的机会；例如一次运行中发现有几个人都在询问如何导出文件，agent 就会创建一个“导出文件太隐蔽”的 signal，并在里面记录哪些用户遇到了这个问题。00:06:30--00:06:50

Signal 的工程定义

signal 是 durable file-backed observation：有来源、有证据、有时间线、可被其他 loop 读取的业务观察。它可以来自客户工单、SEO 数据、广告关键词、产品分析或后端日志。一个好 signal 应该回答三件事：发生了什么、证据在哪里、哪个 loop 应该继续处理。

把 signal 写成文件的价值在于累积。单个客户问“导出在哪里”只是一次客服事件；五个客户都问同一件事，就可能是产品信息架构问题。signal 文件让这些离散事件聚在同一张证据卡上，后续 product loop 可以据此创建 task，support loop 可以继续追加案例，growth loop 可以评估影响范围。

⁶视频画面时间区间：00:06:20–00:06:55。

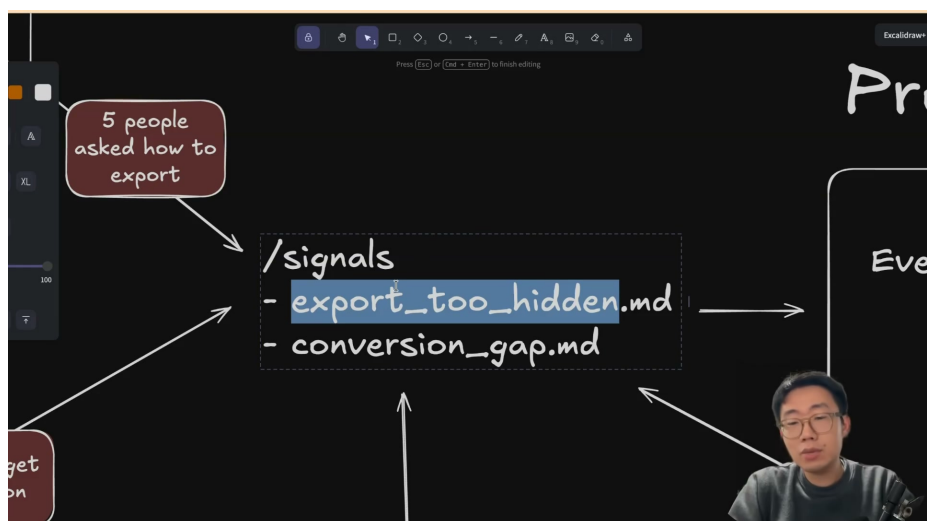


图 6: 反复出现的用户问题会沉淀为 signal 文件，例如“多个用户询问如何导出”可以进入产品改进优先级。⁶

4.2 多个 loop 共享同一个业务大脑

讲者随后列出公司内实际运行的多个 loop: support loop、SEO loop、product growth loop、ads loop。SEO loop 每天上午 9 点抓取数据，研究相关主题，并发布 SEO 页面；在分析过程中，它可能发现某页面点击很多但漏斗转化率不够高，于是把“转化缺口”写入特定路径的 signal。00:06:55--00:07:19

Product loop 过去可能只看 post-hook session 和 analytics 数据来确定优先级；有了共享文件系统后，它在分析数据时还会查看其他 loop 或部门识别出的 signals。00:07:19--00:07:44 这意味着它可以优先修复一个被多次报告的 bug，或者抓住 SEO / 市场团队正在优化的机会。00:07:42--00:07:50

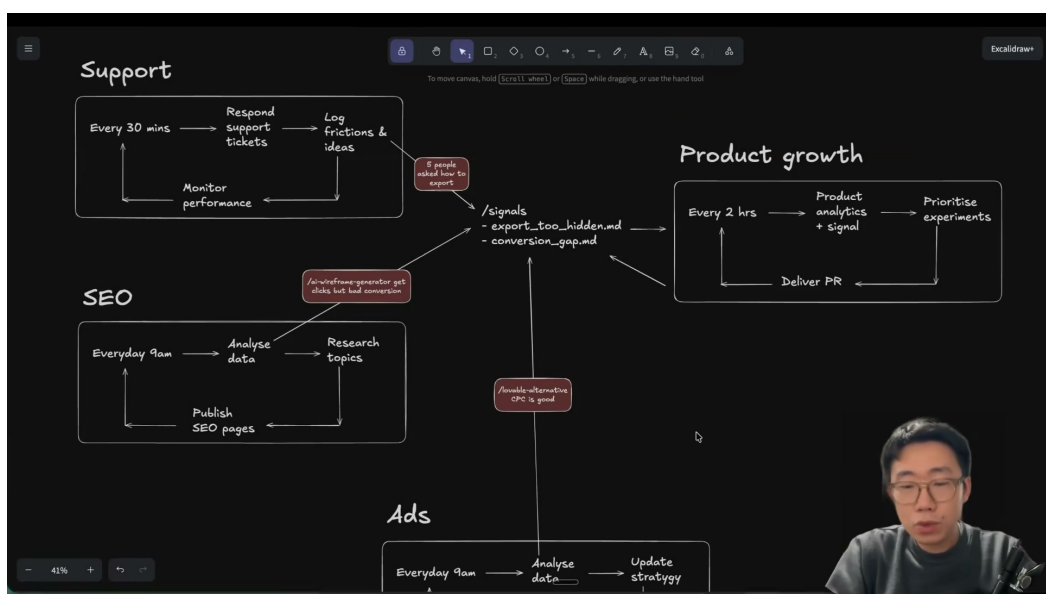


图 7: 多个业务 loop 共享 /signals 后，支持、SEO、广告和产品增长的观察可以互相改变优先级。

共享 brain 的直觉

可以把 `/signals` 理解为团队的业务神经中枢。support loop 像感知客户疼痛的神经末梢，SEO loop 像观察流量和内容机会的传感器，ads loop 像观察付费关键词的传感器，product growth loop 像负责行动选择的运动系统。它们共享一个 signal 层时，局部观察才会变成全局行动。

广告 loop 的例子进一步说明这种交叉增益。如果 ads loop 发现某个关键词点击率不错，而团队尚未围绕它制作自然内容，这个 signal 可以反馈给 SEO loop，使 SEO loop 优先为这个关键词生产自然内容。00:07:50--00:08:04 这里的收益来自信息流动：广告支出发现的机会可以被自然内容复用，客服发现的摩擦可以被产品修复，产品实验结果又能反向影响支持话术和 SEO 内容。

4.3 复利来自共享格式，而不是更多 agent

视频在 00:08:04--00:08:12 总结说，所有这些不同 loop 每小时或每天发生，并共享同一个 brain，这正是复利效应真正开始发挥作用的地方。这里容易产生误解：复利不来自“agent 数量更多”，而来自共享状态和共享格式。十个各自写临时日志的 agent 只会制造十份孤岛；三个遵循同一 signal schema 的 loop 反而可能更有价值。

共享文件夹会放大好结构，也会放大坏结构

`/signals` 如果缺少来源、重复计数、影响范围、状态和相关 artifact 链接，很快会变成噪音池。复利系统要求格式稳定、字段清楚、所有权明确、归档规则明确。否则多个 loop 共享的 brain 会退化成一堆难以信任的碎片。

从工程角度看，signal 至少应该包含标题、来源、证据、影响、状态、相关 artifact、建议 owner 和 timeline。这个结构像数据库表的 schema：字段少一点没关系，字段含义要稳定。稳定 schema 让 support、SEO、ads 和 product growth 能在不同节奏下追加同一类信息，也让 product loop 能排序，而无需每次重新解析一段自然语言。

4.4 四个核心组件：trigger、file structure、tools、verify

讲者在 00:08:18--00:08:58 把落地条件收束为四个组件：第一是 trigger；第二也是最重要的一点，是 file structure 的设计；第三是给 agent 提供不同工具和 connectors；第四是确保代码库或环境允许并行、自主工作，并让每个 agent 能验证自己的工作。

这四个组件可以放在一个简单的系统图里理解：

$$\text{Loop value} \approx f(\text{Trigger, File structure, Tools, Verify})$$

其中 trigger 决定系统何时启动，file structure 决定状态能否留下，tools 决定 agent 能否接触真实事实，verify 决定结果能否被信任。任何一项缺失，loop 都会偏向脆弱：没有 trigger 就靠人工提醒，没有 file structure 就丢状态，没有 tools 就只能猜测，没有 verify 就无法区分完成和幻觉。

⁷ 视频画面时间区间：00:06:55–00:08:12。

⁸ 视频画面时间区间：00:08:20–00:08:58。

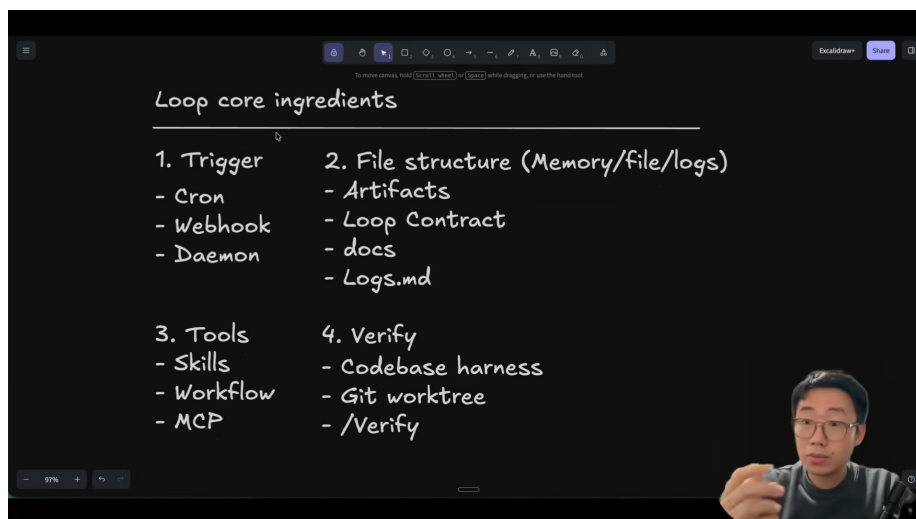


图 8: 构建 loop 的四个核心组件：触发器、文件结构、工具能力和验证机制。⁸

复利 loop 的核心机制

复利 loop 的关键在于多个 loop 把局部观察写入共享 signal 层，并用稳定 artifact、contract、tools 和 verification 把这些观察转成下一轮行动。系统越运行，信号越集中，优先级越清楚，下一轮 agent 的起点越高。

4.5 本章小结

本章说明了多个业务 loop 产生复利的条件。Support loop 把客户摩擦写入 signals；SEO loop 把流量和转化缺口写入 signals；ads loop 把关键词机会写入 signals；product growth loop 读取这些共享观察来调整优先级和交付 PR。这个系统成立的前提，是 trigger、file structure、tools/connectors 和 verify 四类组件同时存在。下一章将进入第一个实践组件：如何把代码库本身建设成 agent 能读懂、跑起来、验得准的 Codebase Harness。

5 Codebase Harness：让 agent 看得懂、跑得起、验得准

前一章已经说明，多个 loop 可以通过共享 signal（信号）和 artifact（工件）形成复利。本章进入更硬的工程底座：代码库本身必须适合 agent 工作。视频在 00:08:44–00:08:56 先把问题说得很直接：业务要启动复利 loop，除了 trigger、file structure、tools/connectors，还需要一个 agent-friendly environment，使许多 agent 能并行、自主地工作，并验证各自的结果。这里的关键词是 Codebase Harness，也就是围绕 repository / runtime environment 建出的 agent harness（运行马具）。

Codebase Harness 的工作定义

Codebase Harness 是代码库周围的一组运行支撑：文档入口、规则检查、本地运行脚本、并行 worktree 约定、状态跳转脚本、浏览器验证、E2E tests、PR 流程与只读 review agent。它的目标很朴素：让 agent 快速知道改哪里，低成本跑起来，留下可复查的 verification evidence。

$$H_R = I_{\text{docs}} + C_{\text{checks}} + D_{\text{dev}} \\ + W_{\text{worktree}} + S_{\text{state}} + V_{\text{verify}}$$

H_R ：面向代码库 R 的 harness； I_{docs} ：入口文档与索引； C_{checks} ：自定义检查； D_{dev} ：本地开发脚本； W_{worktree} ：并行工作目录约束； S_{state} ：状态跳转脚本； V_{verify} ：测试、浏览器证据与独立验收。

这个公式只是一张清单。它表达的工程判断是：loop 的质量受 agent 能否稳定操作代码库影响。一个优秀模型进入混乱仓库，会像经验丰富的司机被丢进没有路牌、没有维修站、没有测速仪的赛道；模型可能很强，系统仍会持续制造返工。

5.1 可读代码库：把上下文工程写进仓库

视频在 00:09:04–00:09:18 把代码库的第一条要求定义为清晰易读：agent 应当轻松理解“在哪里改什么”，并且能在本地启动开发服务器、使用验证工具。所谓 readable repo，评估重点落在仓库知识是否可发现、可更新、可执行；目录名漂亮程度只是表层。

最小做法是设置一个入口文件。视频提到 OpenAI 把类似 AGENTS.md 的项目指令文件做成大约 100 行的索引，指向各种文档系统，供 agent 逐步发现信息（00:09:20–00:09:30）。在不同工具链中，这个入口也可能叫 CLAUDE.md、Codex.md 或项目自己的 handbook。名字可以变，职责要稳定：

- 告诉 agent 仓库的主要模块、边界和术语。
- 指向更细的设计文档、API 约定、测试说明和发布流程。
- 记录“先读什么、再跑什么、最后提交什么”的工作顺序。
- 把容易误用的旧路径、弃用模块和危险操作写清楚。

这类入口文件的价值在于降低上下文搜索成本。agent 每次开工都像新成员入职，如果它需要靠全文搜索猜测架构，有限的有效上下文会被噪声吃掉。文档索引相当于给它一张楼层导览图：先知道电梯在哪里，再去找具体房间。

Indexed docs 与 Context Engineering

Context Engineering 在代码库里也发生在 prompt 之外的文件系统中。把文档做成层级索引，本质上是在文件系统中实现上下文路由：短入口负责定位，长文档负责细节，检查脚本负责把硬规则变成机器可见的反馈。这样 agent 使用上下文窗口时，会优先读取和任务相关的局部材料。

视频随后补上一个更强的手段：custom checks。00:09:36–00:10:01 的例子是复杂 monorepo 中存在一些不希望 agent 使用的旧文件夹；当 agent 新写文件从这些旧目录导入时，检查会直接报错。

⁹ 视频画面时间区间：00:10:01–00:10:14。

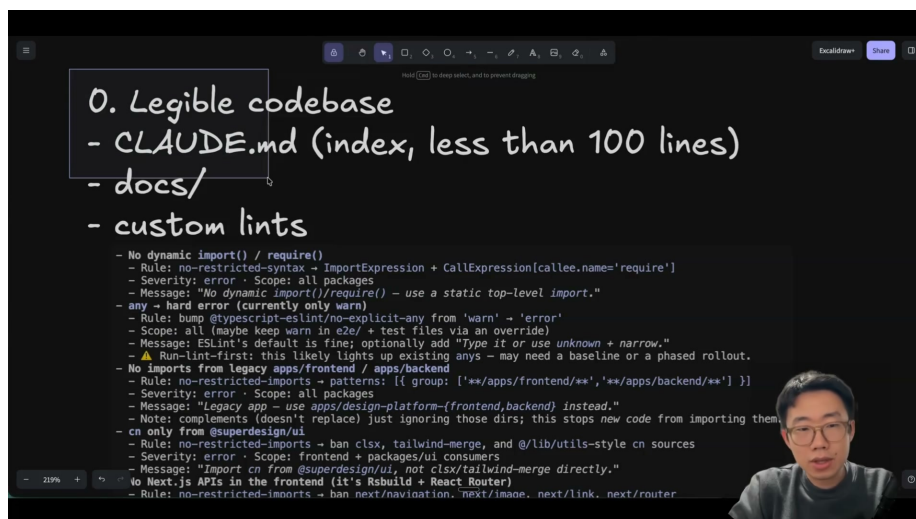


图 9: 可读代码库的第一层治理: 短入口文档、分层文档目录和自定义 lint 规则。⁹

这里需要纠正 ASR 的含混表达: 视频强调的做法, 是把规则注入程序化检查, 让错误自动浮现, 减少 agent 独自搜索相关信息的压力。

自定义检查适合承载三类信息: 第一类是架构边界, 例如前端层不能直接访问后端内部模块; 第二类是迁移规则, 例如旧 SDK、旧组件库、旧目录禁止新增引用; 第三类是业务安全线, 例如支付、权限、数据导出必须通过指定接口。规则越关键, 越应该从口头约定升级为可运行检查。

5.2 可执行环境: 让本地启动成为一条命令

可读只能解决“看得懂”。loop 真的跑起来, 还需要 executable repo。视频在 00:10:14–00:10:33 说, agent 应当在开发服务器已经运行的情况下开始工作, 理想情况下无需消耗太多 token 或认知负荷; 他们的代码库写了本地开发脚本, agent 运行该脚本即可启动整个 dev server。

这一步看似琐碎, 实际很致命。许多 agent 任务失败, 常见原因是环境启动要经过七八个隐含步骤: 装依赖、设置环境变量、启动数据库、迁移 schema、打开 mock 服务、登录测试账号、再启动前端。模型即使理解需求, 也会被这些隐含步骤卡住。人类老员工记得这些步骤, 新 agent 没有这种肌肉记忆。一个 `dev-local.sh`、`dev-local.ps1` 或 `npm run dev:local`, 相当于把“老员工经验”压缩成可执行入口。

接着是 worktree-friendly parallelism。字幕在 00:10:33–00:10:46 说, 当有五个不同的并行 agent 时, 它们都在各自的 worktree 上工作, 仍然可以启动开发服务器并测试, 互相不冲突。对主要有 Python 背景的读者, 可以把 Git worktree 理解成“给每个实验开一个独立工作副本”: 共享同一个 Git 仓库历史, 每个副本有自己的文件树和分支。这样 agent A 修支付流程, agent B 修注册页, agent C 做文档迁移, 三者不会争抢同一个工作目录。

Worktree 友好意味着什么

一个 worktree-friendly repo 至少要处理端口、缓存、临时文件、数据库实例和环境变量的隔离。否则五个 agent 同时运行时, 常见结果是端口冲突、测试数据互相污染、生成文件互相覆盖。真正适合 loop 的环境, 需要让并行成为默认路径, 避免停留在临时技巧。

视频还强调 state-jump scripts: 最好提供一些脚本, 让 agent 快速跳到已授权、未授权、特定测

¹⁰ 视频画面时间区间: 00:10:14–00:11:01。

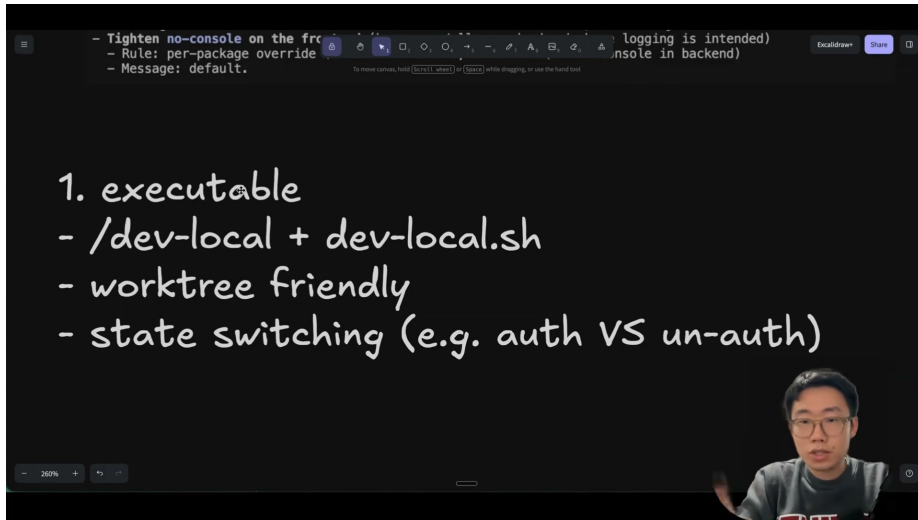


图 10: 可执行环境需要本地启动脚本、worktree 友好设计, 以及可复现的状态切换脚本。¹⁰

试场景等状态 (00:10:47–00:10:59)。这类脚本像游戏里的存档点。测试“升级流程”时, agent 不必每次从注册、登录、填资料、绑定支付开始跑完整前戏; 脚本可以把应用切到目标状态, 然后 agent 专注验证关键行为。

5.3 可验证性: 证据优先于自我感觉

Codebase Harness 的第三根柱子是 verification。视频在 00:11:01–00:11:07 说, 需要给 agent 合适工具去实际测试并记录结果。这里的“记录”很重要: 验证结果要能被人类、PR reviewer、后续 agent 复查。没有证据链的“我检查过了”, 在循环系统里会不断放大风险。

视频给出的第一类工具是 Playwright client。00:11:07–00:11:19 提到, 它能让 agent 有效使用浏览器, 录制视频片段, 并把片段上传到 GitHub pull request。这个点的工程含义很清楚: 浏览器自动化同时承担操作和 evidence 生产两种职责。一个 PR 附带“登录成功、升级成功、错误提示正确显示”的视频或截图, review 成本会明显下降。

第二类工具是关键路径的 E2E tests。视频列举了 upgrade flow、signup flow, 以及在 Super Design 案例中有效创建设计的流程 (00:11:19–00:11:32)。E2E test 的价值在于覆盖用户真正经过的路径: 前端、后端、鉴权、支付、数据库、异步任务连在一起, 任何一段断了都可能让功能失效。单元测试像检查零件尺寸, E2E 测试像把整辆车开上路。

弱验证会把 loop 变成缺陷放大器

让实现 agent 自己验收自己的工作, 会放大盲点。agent 已经沿着某条推理路径完成实现, 它最容易相信同一条路径上的检查也足够。视频在 00:11:40–00:11:52 明确强调 PR skill 会根据详细规范生成只读验证智能体。只读 review agent 无法修改代码, 只能检查结果、比较规范、运行验证、指出缺口, 因此更适合承担独立验收。

这里有一个容易被低估的风险: self-verification 会让错误带着“已验证”的标签进入下一轮 loop。下一次 trigger 唤醒 agent 时, 它会把这个标签当成可靠历史, 继续在错误基础上分配任务。数学上可以把它理解成误差递推: 若每轮验证可靠性为 p , 系统连续 n 轮都依赖同一套弱验证时, 整体可

¹¹ 视频画面时间区间: 00:11:38–00:11:52。

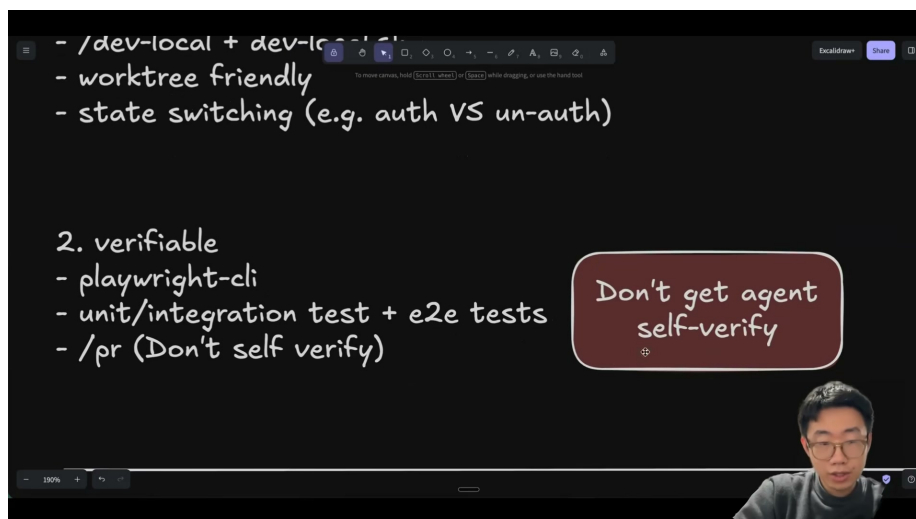


图 11: 验证层由 Playwright、单元/集成/E2E 测试、PR 技能和独立 review agent 共同构成。¹¹

信度不会自然上升；如果验证盲点相关，误差会沿着 loop 传播。工程上更直接的说法是：自动化速度越快，验证门槛越要硬。

PR skill 的作用，是把提交前动作标准化。它可以要求 agent 跑 lint、类型检查、单元测试、E2E tests，附 Playwright evidence，再启动 read-only review agent。这样 PR 会从代码差异扩展成一套可复查的验收包。视频在 00:11:32–00:11:38 说 PR skill 定义了提交 pull request 前必须执行的步骤列表，这正是把“好习惯”变成 harness 的例子。

5.4 从代码库设置到 Loop Engineering

这一章的结尾，视频把 codebase harness 上升为第一件该做的事。00:11:53–00:12:09 介绍了一个 setup codebase harness skill，可交给 Claude Code 或 Codex，用来设置有用的脚本、文档和技能；00:12:20–00:12:30 又强调，应先让代码库处于 agent 可以验证工作的状态，即使暂时没有写 loop，这件事本身也很有帮助。

这个顺序很现实。没有 readable repo，agent 会把时间浪费在猜测架构上；没有 executable repo，agent 会把任务卡在启动环境上；没有 worktree-friendly 设计，并行 agent 会互相干扰；没有 verification evidence，团队无法判断自动化产出的可信度。Loop Engineering 的外层 trigger、artifact 和 logs 需要落在这样的底座上，才有机会稳定复利。

5.5 本章小结

Codebase Harness 是 agent 能否在真实代码库中长期工作的基础设施。它包含 indexed docs / AGENTS.md、custom checks、本地 dev script、worktree-friendly parallelism、state-jump scripts、Playwright evidence、E2E tests、PR skill 与 read-only review agent。

本章最关键的判断有三点。第一，可读性要落实为索引文档和机器检查，帮助 agent 在有限上下文中找到正确材料。第二，可执行性要落实为一条命令启动和可隔离并行，让多个 agent 同时工作时仍可测试。第三，可验证性要落实为证据、关键路径测试和独立只读验收，避免 self-verification 把错误带入下一轮 loop。后续章节讨论 artifact、loop contract 和 global work log 时，都默认这块代码库底座已经存在。

6 文件、契约与日志：把 loop 变成可维护系统

前一章讲的是 codebase harness：代码库要清晰、可执行、可验证，agent 才有稳定工作的地面。这里进入第二层问题：当多个 loop 长期运行时，它们怎样记住自己做过什么、怎样把观察交给其他 loop、怎样避免每次启动都从零猜上下文？讲者在 00:12:30–00:15:54 给出的答案很朴素：把循环中的知识落到文件系统里，用 artifact、loop contract 和 global work log 三类文件抽象承接状态。

Loop Engineering 在这一段里的重点已经从单次 agent 执行转向维护性。一个一次性脚本可以把结果打印到终端；一个每天、每小时、每 30 分钟运行的 loop 需要留下可读、可追踪、可复用的文件。直觉上，这像团队里的工单系统和项目文档：人可以忘记，聊天窗口会结束，文件系统会让下一位接手者知道“这件事现在到哪一步了”。

6.1 Artifact：agent 工作留下的结构化产物

Artifact 的定义

artifact（工件）是 agent 在调查、处理、总结或交付过程中留下的结构化产物。视频中的例子包括 docs、signals、tasks、campaigns、tickets 等类型（00:12:39–00:12:58）。它们共同构成一个共享知识层，让后续 loop 可以读取、补充、引用和追踪。

讲者把 artifact 描述成“每个智能体工作或发现的产出”，并强调它像共享知识层（00:12:39–00:12:45）。字幕中有若干同音误识别，本文统一按画面和上下文更正为 artifact。广告 campaign 可以是 artifact，support ticket 可以是 artifact，产品 task 也可以是 artifact。判断重点在于每类 artifact 是否有稳定边界：归属范围、外部链接、更新流程和新增条目规则都要写清楚。

这类边界通常写在每个 artifact 文件夹的 README.md 里。讲者提到，在 artifact folder 中会定义 README，说明归属范围、添加新项目的流程，以及该工件的 schema（00:13:11–00:13:23）。如果拿 Python 项目类比，artifact schema 像一个 dataclass 或 Pydantic model：字段不需要复杂，字段含义要稳定。一个 ticket artifact 至少能记录用户、问题、处理状态、证据链接和后续动作；一个 campaign artifact 至少能记录目标、投放渠道、指标、调整历史和结论。

讲者给出的最小结构可以压缩成三层（00:13:26–00:13:34）：

- **metadata**：前置信息和来源，例如客户、渠道、时间、负责人、状态、相关链接。
- **body**：内容本身，例如问题描述、分析过程、回复草稿、产品建议或 campaign 说明。
- **timeline**：该 artifact 的所有重要变更，记录谁在什么时候做了什么、为什么做。

为什么文件结构比聊天记录更适合 loop

聊天记录像临时白板，适合推理过程；artifact 像项目档案，适合跨会话复用。loop 每次启动时需要的是“当前状态、证据、下一步”，而无需重读一次完整对话。把 artifact 固化成文件后，agent 可以用搜索、读取、追加、链接等普通工程动作来维护知识。

可以用一个小公式描述本章和前几章的关系：

$$L_i = (T_i, G_i, A_i, H, S, V_i)$$

其中：

¹²视频画面时间区间：00:14:20–00:15:12。

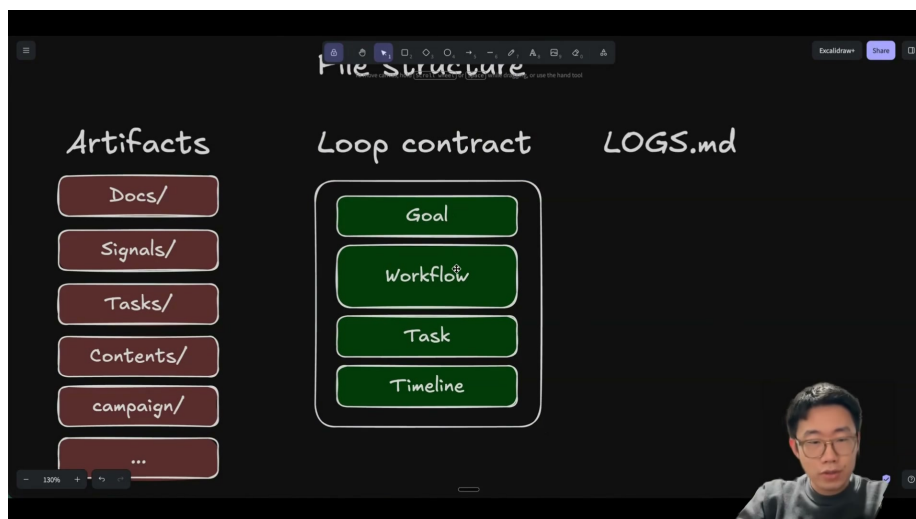


图 12: 文件结构三件套: Artifacts 保存产物, Loop contract 保存目标/流程/任务/时间线, LOGS.md 汇总跨循环记录。¹²

- L_i 表示第 i 个 loop, 例如 support loop、SEO loop、ads loop。
- T_i 是 trigger, 决定这个 loop 在何时被唤醒。
- G_i 是 goal, 说明它要优化什么业务结果。
- A_i 是执行调查和行动的 agent。
- H 是共享 harness, 包括文件系统、工具、脚本、日志和验证环境。
- S 是 signal 集合, 即多个 loop 共同读写的业务观察层。
- V_i 是 verification evidence, 表示这个 loop 产生结果后的验证证据。

这个公式的直觉是: 一个可维护 loop 需要触发器和目标, 也需要共享状态与证据。缺少 artifact 的 loop 像没有病例档案的医生, 每次看诊都要重新问一遍病史, 时间久了必然丢信息。

6.2 Signal: 多个 loop 共享的业务神经末梢

Signal 的定义

signal (信号) 是 loop 观察到的可复用业务线索: 产品摩擦、客户反馈、异常模式、增长机会、漏斗缺口、关键词机会等。合格的 signal 要避免泛泛想法, 必须有来源、证据、时间线, 并能链接到相关 ticket、doc、task 或 campaign (00:13:34-00:13:51)。

signal 也是一种 artifact, 但它值得单独命名。原因在于 signal 负责把不同 loop 的发现接到同一个业务神经系统上。support loop 看到多个用户都找不到导出入口, 会写下“导出功能过于隐蔽”的 signal; SEO loop 看到某个页面点击很多、转化偏低, 会写下“某路径存在转化缺口”的 signal; ads loop 看到某个关键词点击率不错, 会把它反馈给 SEO loop。每个 signal 都应能说明: 它从哪里来、重复出现了几次、影响了哪些用户或指标、建议哪个 loop 继续处理。

讲者在 00:13:43-00:13:49 强调, signal 可以链接不同来源, 详细记录原始客户反馈, 甚至链接支持工单 artifact。这个链接关系很关键: 没有来源的 signal 只是灵感, 有来源的 signal 才能进入

优先级排序。对于产品团队来说，signal 像一张小型证据卡：一面写观察，一面贴证据。证据足够多时，product loop 可以把它转成 task；证据不足时，它可以留在观察池里继续累积。

没有共享 artifact 的 loop 会形成信息孤岛

如果 support、SEO、ads、product growth 各自写自己的临时文件，后果会很硬：support 看到的客户摩擦进不了产品优先级，ads 看到的关键词机会传不到 SEO，工程修复也无法反向告诉客服。Loop Engineering 的复利来自可共享的 signal 层；文件格式混乱时，复利会变成噪音。

讲者还提到可以为 artifact 构建小型微应用，用来浏览产品信号，并让每个信号链接到另一个相关工件（00:13:58–00:14:12）。这一步的真正价值是降低人类介入成本。人类负责人无需阅读所有 loop 的运行记录，只要查看 signal 面板，就能知道哪些事情需要关注、哪些已经有后续 task、哪些已经被某个 agent 处理。

6.3 Loop contract：每个 domain loop 的 README

Loop contract 的定义

loop contract（循环契约）是每个 domain loop 的规范文件，通常表现为 loop 文件夹里的 README.md。它写清目标、工作流程、边界、todo 列表和 timeline，使下一次触发的 agent 能读取上下文并选择下一步最佳行动（00:14:27–00:15:11）。

artifact 解决“产物怎样存”，signal 解决“观察怎样共享”，loop contract 解决“这个 loop 到底怎样运行”。讲者说，每个循环本身通常会有一个契约，包含目标、流程、待办事项列表、任务优先级调整方式和时间线（00:14:28–00:14:45）。这类似工程团队里的 runbook：值班工程师不需要问上一位同事所有细节，读 runbook 就知道服务目标、排障流程、升级边界和最近事件。

一个 support loop contract 至少应该回答这些问题：

- **Goal**：这个 loop 要减少什么摩擦？提高什么响应质量？是否允许直接回复客户？
- **Workflow**：每轮先拉取哪些工单，怎样 triage，怎样查支付、订阅、权限和后端日志。
- **Boundary**：哪些情况只能草拟回复，哪些情况可以直接处理，哪些需要人工确认。
- **Todo**：上一次遗留了哪些待处理项，哪些需要下轮优先继续。
- **Timeline**：本领域最近发生了什么，哪些客户问题已解决，哪些工程任务已创建。

Contract 是给 agent 的长期接口

如果把一个 loop 看成函数，trigger 像调用时机，tools 像依赖注入，contract 就像函数签名和文档。它告诉 agent 输入是什么、允许做什么、输出应写到哪里、哪些副作用需要记录。没有 contract 时，agent 每轮都会凭临时上下文猜边界。

这一点也是“契约”一词的意义：它约束 agent，也保护系统。agent 可以自主工作，因为它知道可执行边界；人类可以放手，因为边界被写成文件，下一轮还能复查。讲者在 00:15:01–00:15:11 说，每当循环触发时，agent 会读取契约，了解目标、工作流程和之前发生的事情，再采取下一步最佳行动。换句话说，contract 是 loop 的启动上下文。

6.4 Global work log: 组织记忆与临时上下文

讲者在第三类文件中引入 global work log (全局工作日志)。乍看它和 artifact timeline、loop contract timeline 有重叠, 所以讲者主动解释: 日常工作常常混合了两类状态, 一类是审查 loop 输出, 一类是在副驾驶式协作中与 agent 处理困难或创意任务 (00:15:12–00:15:33)。这些信息未必属于某个稳定 artifact, 也未必只属于单一 domain loop, 但它们会影响下一轮判断。

Global work log 的定义

global work log 是全局 Markdown 日志, 记录跨 domain 的临时信息、人工协作进展、重要决策和 agent 完成的大块工作。每个 agent 完成大量工作后写入日志, 开始新工作前读取最近 5–10 条记录 (00:15:40–00:15:52)。

这里需要谨慎处理 ASR。字幕里的 “lg man” 在后文更像某个 log Markdown 文件, 但未经过画面确认时不应强行命名为具体文件。写作中称为 global work log 或全局工作日志即可。核心机制已经明确: 写入最近工作、读取最近 5–10 条, 帮助 agent 先了解不同领域正在发生什么 (00:15:35–00:15:52)。

global work log 和 timeline 的分工可以这样理解:

- artifact timeline 记录某个具体产物怎样变化。
- loop contract timeline 记录某个 domain loop 最近发生了什么。
- global work log 记录跨领域、跨会话、暂时还没有稳定归属的工作背景。

这三层像地图的不同缩放级别。artifact timeline 是街道级, loop timeline 是城区级, global work log 是全城交通播报。agent 每次启动时先看全城播报, 再进入某个城区 runbook, 最后打开具体街道档案。

日志会成为决策输入, 所以噪音会被放大

global work log 不能变成随手堆放的聊天垃圾场。它应记录事实、决策、证据链接、未解决问题和下一步建议。若日志里充满含糊判断, 后续 agent 会把这些含糊判断当成真实上下文继续传播。

6.5 本章小结

本章把 Loop Engineering 的文件层拆成三类: artifact 保存结构化产物, signal 保存跨 loop 共享的业务观察, loop contract 保存每个 domain loop 的目标、流程、边界、todo 和 timeline, global work log 保存跨领域的临时上下文与组织记忆。讲者真正想强调的是, loop 的价值来自持续运行; 持续运行需要可维护状态; 可维护状态要落在能被 agent 和人类共同读取的文件系统中。下一章会把这些抽象压到一个具体例子上: 如何搭建第一个 support loop。

7 落地示例: 搭建第一个 support loop

在定义了 artifact、signal、loop contract 和 global work log 之后, 讲者用 support loop 作为第一个落地案例 (00:15:54–00:20:24)。这个选择很实际: 支持工单天然有触发频率、外部事实、客户

反馈、可验证结果和产品改进线索。一个 support loop 如果设计得好，可以同时回复客户、发现产品摩擦、创建工程 task，并把这些信息写回共享 artifact 系统。

本章的目标是把抽象架构变成实施步骤。support loop 的基本目标是：每 30 分钟或每小时拉取最近工单，读取业务和系统背景，调查问题，起草或发送回复，记录 ticket artifact、signal、工程 task 和日志，最后在 contract 中留下下一轮可接手的状态。

7.1 先定义 loop 要处理的真实工作

讲者先给出一个假设：让 agent 每隔 30 分钟提取最近所有支持工单，在信息充分时起草回复或直接回复客户，并记录它发现的摩擦点和想法 (00:15:59–00:16:11)。这就是 support loop 的最小闭环。它要超出客服后台礼貌文本生成，连接业务系统、支付系统、数据库、后端日志和产品反馈。

Support loop 的工作定义

support loop 是一个以客户工单为 trigger 输入的 domain loop。每轮它读取工单、trriage 问题、查询必要系统、生成回复或后续任务，并把 ticket、signal、task 与日志写回共享文件结构。它的成果既包括客户可见回复，也包括公司内部可复用的产品和工程信息。

讲者在自己的案例中要求 loop 获取过去 X 小时内更新的工单，调查用户提到的问题，创建 ticket artifact 来记录处理过的工单，必要时创建工程工单记录反馈，最后记录自己做过的一切 (00:16:43–00:17:01)。这里的 X 是可配置窗口：测试阶段可以设为 1 小时，稳定后可以按工单量调成 30 分钟、1 小时或更长。

为什么 support loop 适合作为第一个 loop

支持工单有清晰输入、清晰证据和清晰反馈。客户说了什么是输入，Intercom、Stripe、Supabase、Render 等系统给出事实证据，回复质量和后续客户反应给出反馈。相比“让 agent 自主增长业务”这种大目标，support loop 更容易手动校准，也更容易发现系统边界。

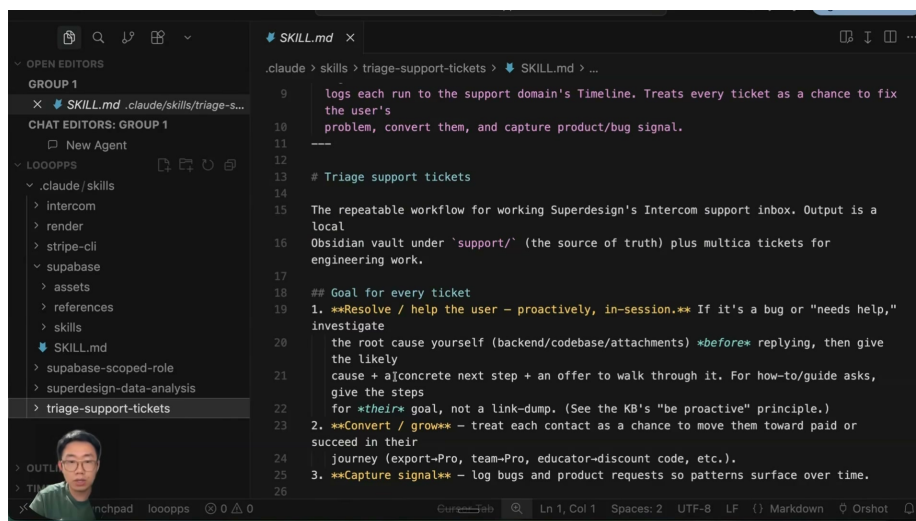


图 13: 支持循环所需的技能集合：Intercom、Stripe、Supabase、Render 与工单分流技能把业务动作转成 agent 可调用能力。¹³

¹³ 视频画面时间区间：00:16:11–00:16:41。

7.2 准备 skills 和 connectors

support loop 第一件事是给 agent 准备可调用的 skills 和 connectors。讲者列出的工具很具体：Intercom 用来获取支持工单，Stripe 用来检查支付和订阅数据，Supabase 用来检查访问权限和调试支付相关信息，Render 用来获取后端日志，还需要一些脚本帮助 agent 提取信息，以及一个用于 triage 支持工单的 skill (00:16:11–00:16:38)。

这组工具覆盖了客服排障的关键事实源：

- **Intercom**：读取客户对话、工单状态、历史消息和回复上下文。
- **Stripe**：检查订阅、付款、退款、发票、试用期和支付失败。
- **Supabase**：检查用户权限、账号状态、数据库记录和产品内访问问题。
- **Render**：读取后端日志、部署状态和服务错误。
- **Triage skill**：把工单分成可直接回复、需调查、需工程介入、需人工确认等类别。

工具权限是支持循环的硬边界

support loop 可能触达客户数据、支付数据和后端日志。权限应该按最小必要范围发放，并在 contract 中写清哪些动作允许自动执行，哪些动作只能草拟建议。尤其是退款、订阅变更、账号权限调整和客户承诺，通常需要更高的人类确认门槛。

这些 skills 把事实查询变成稳定动作，同时保留权限边界。一个没有 Stripe 权限的 support agent 只能猜支付状态；一个没有 Render 日志的 support agent 只能复述用户报错；一个没有 triage skill 的 support agent 很容易把工程漏洞当作普通客服话术处理。

7.3 写入业务背景文件和工程规则

讲者接着强调，通常还要创建一个面向 Claude Code 或 Codex 的 business-background file (业务背景 Markdown 文件)，让 agent 对业务有深入了解 (00:17:03–00:17:20)。这个文件可以是项目里的 CLAUDE.md、AGENTS.md、Codex.md 或同类入口文档，具体命名取决于使用的 agent 工具链。它应保存产品是什么、用户是谁、主要流程是什么、常见问题有哪些、哪些系统是事实源。

在支持场景中，业务背景文件至少应包含：

- 产品定位、核心用户、付费层级和常见使用路径。
- 支持工单分类，例如支付、登录、权限、导出、性能、bug、功能请求。
- 客户沟通风格，例如回复语气、是否允许承诺修复时间、何时升级给人工。
- 数据系统索引，例如 Intercom、Stripe、Supabase、Render 分别查什么。
- 工程协作规则，例如必须使用 Git worktree、如何创建工程 task、如何留下验证证据。

讲者还会放入生成工程 agent 的规则，解释代码库结构，并要求每次使用 Git worktree，同时写入如何管理 worktree 的契约 (00:17:20–00:17:33)。对主要有 Python 背景的读者，可以把 Git worktree 想成给每个实验开一个独立目录副本：多个 agent 可以各自在自己的副本里改代码、跑测试，最后再把可接受的改动合并回来。

业务背景文件和 loop contract 的分工

业务背景文件描述公司和代码库的长期事实；loop contract 描述某个 domain loop 的目标、流程和边界。前者像公司手册，后者像某条生产线的操作规程。support loop 启动时通常两者都要读：先理解业务，再执行本领域流程。

7.4 用 architecture markdown 生成 artifact 和 domain 框架

除了业务背景文件，讲者还提到一个 architecture markdown 文件。它被业务背景文件引用，作为通用结构指令：要求 agent 定义不同的 artifact 类型、不同 loop domain 的指令，以及 logs 的规范（00:17:38–00:17:54）。这一步把上一章的抽象文件层变成项目模板。

在讲者的示例里，他只需要指向这个 architecture markdown，说“帮他设置相关 artifacts”：signals、tasks、tickets、docs，再加上一个 domain agent。agent 读取架构文件后，就能搭建出文档、signal、ticket 等 artifact 类型（00:17:54–00:18:19）。这里“artifact 类型”可以理解成项目里的目录和 schema 组合，例如：

- artifacts/docs/：长期说明、流程文档、客户可见或内部文档。
- artifacts/signals/：客户反馈、产品想法、摩擦点、观察结果。
- artifacts/tickets/：支持工单处理记录、回复草稿、调查过程。
- artifacts/tasks/：工程 task、修复建议、产品实验任务。
- domains/support/：support loop 的 contract、todo、timeline 和运行记录。
- 全局工作日志：记录跨 domain 的重要工作与最近上下文。

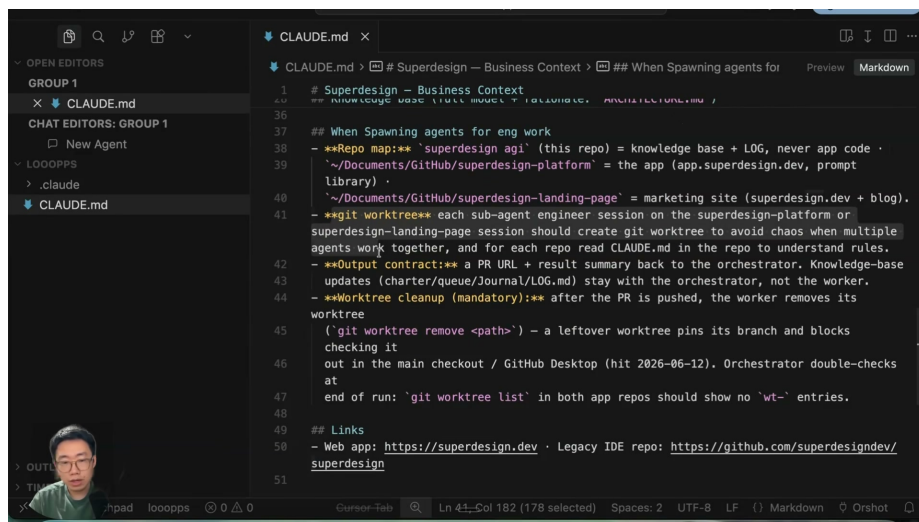


图 14: CLAUDE.md 可以把 repo map、worktree 规则、输出契约和收尾检查写成持久约束。¹⁴

¹⁴视频画面时间区间：00:17:20–00:17:36。

Domain contract 的价值

domain contract 把 support loop 的目标、整体 workflow、边界、未完成任务列表和 timeline 写成稳定文件。它让 agent 每次运行时先读规范，再处理工单；也让人类可以审查 loop 的真实行为，避免只看 agent 的最终回复。

讲者在 00:18:22–00:18:35 提到创建 `domains/support` 文件夹，并为每个循环定义契约，用来捕捉目标、整体工作流、边界、未完成任务列表和时间线。这个文件是 support loop 的中枢。没有它，agent 每次都要从新 prompt 猜流程；有了它，agent 可以不断把测试运行和正式运行的经验写回 contract。

7.5 先手动测试，再设置 trigger

讲者的实施顺序很重要：通常先和 agent 手动运行一次，作为 test run；测试运行结束后，再让它创建 contract 并设置 loop (00:18:37–00:18:47)。这一步是把风险压低的关键。自动化之前先人工校准，能发现权限不足、字段缺失、回复风格不合适、artifact schema 不顺手、工程升级边界含糊等问题。

手动测试的 prompt 可以拆成一组明确动作：

1. 获取过去一小时的支持工单 (00:18:48–00:18:52)。
2. 审查并起草回复，把结果保存到 ticket artifact (00:18:52–00:18:55)。
3. 为产品想法和用户摩擦点保存 signal (00:18:55–00:18:58)。
4. 对明显 bug 生成工程 agent 或工程 task (00:18:58–00:19:04)。
5. 记录本轮处理结果、发现的反馈、工程漏洞和未完成事项 (00:19:08–00:19:22)。

讲者说，这就是与 agent 校准的过程：通过查看它处理了几个工单、为每个工单创建了 artifact、识别了客户反馈或摩擦点、记录了明显工程漏洞，人类可以判断 workflow 是否符合要求 (00:19:08–00:19:27)。校准完成后，再让 agent 创建规范的 README contract，并把 loop 设置成每小时触发一次 (00:19:29–00:19:47)。

自动 trigger 应该晚于手动校准

如果手动运行还不能稳定产出 ticket、signal、task 和日志，定时触发只会更快地产生混乱。一个 support loop 至少要通过一次人工验收：回复是否可靠、证据是否完整、artifact 是否可读、工程升级是否守边界、日志是否能帮助下一轮接手。

这里也解释了 30 分钟和每小时两个时间粒度的关系。讲者先用“每隔 30 分钟”描述支持工单拉取的理想循环 (00:15:59–00:16:04)，后面在测试 prompt 中说每小时运行一次并先做测试运行 (00:19:04–00:19:07)。实际落地时应按业务节奏选择：工单量大、响应要求高时用 30 分钟；早期校准阶段或工单量较小时用每小时；无论哪种频率，都应先保证 contract 和 artifact 输出质量。

¹⁵ 视频画面时间区间：00:18:47–00:19:25。

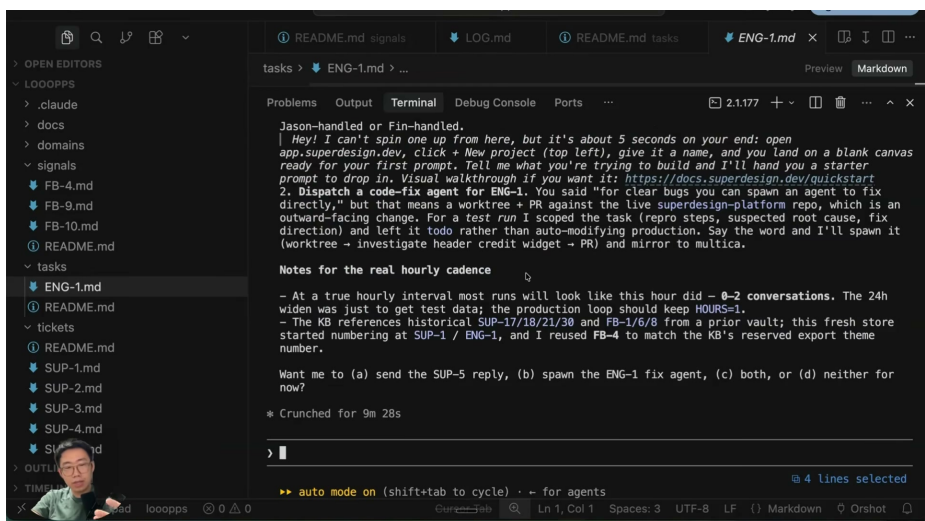


图 15: 测试运行后的产物形态: tickets、signals、tasks 与工程任务共同记录一次支持循环的真实输出。¹⁵

7.6 把模板仓库当成可复制的起点

视频结尾回到实践资源。讲者说他创建了一个名为 `loop engineer setup` 的 repository template, 封装团队设置这些 loop 的最佳实践, 希望读者可以复制文件夹, 指示 agent 设置 artifacts 并启动新的 loops (00:19:48–00:20:04)。这个结尾承担落地提示的功能: 先有一个文件结构模板, 再让 agent 根据业务生成 domain loop。

从工程角度看, 模板仓库的价值在于减少第一次搭建时的空白页问题。它可以预置:

- artifact 目录和 README 模板;
- signal、ticket、task、doc 的 schema 示例;
- domains/support 的 contract 模板;
- global work log 的写入格式;
- skills/connectors 的接入说明;
- manual test run 的验收清单;
- trigger 设置前的风险检查项。

讲者还提到相关资源可以免费使用, 并有数小时逐步演示, 展示如何从零真实设置这些循环, 以及十多个小时关于构建 agent、生产环境和 AI coding 的深入讨论 (00:20:04–00:20:17)。写入 PDF 时保留这部分 substantive close, 因为它指向读者的后续实践路径; 常规告别语则不需要纳入正文。

7.7 本章小结

搭建第一个 support loop 的顺序很清楚: 先定义每轮要处理的真实工单工作, 再准备 Intercom、Stripe、Supabase、Render 和 triage skill 等事实源与工具; 然后让 agent 研究业务, 写入 Claude/Codex business-background file (业务背景文件), 并通过 architecture markdown 生成 artifacts、domains 和 logs 的结构; 接着在 domains/support 下建立 domain contract; 最后先手动

跑一次并校准，产出可接受的 ticket、signal、task 和日志后，再设置每小时或 30 分钟触发的循环。这个例子把全片主线收束到一句工程判断：先设计环境、文件和验证路径，再让 agent 持续工作。

8 总结与延伸：Loop Engineering 的工程判断

全片的主线可以压缩成一个判断：AI agent 的高价值用法正在从一次性 Prompt Engineering 推进到持续运行的 Loop Engineering。Prompt Engineering 解决“这次调用怎样说清楚”，Context Engineering 解决“这次任务怎样给对上下文”，agent harness 解决“agent 运行时怎样拿到工具、状态和验证”，Loop Engineering 继续追问一个更系统的问题：整个 agent 系统下一步该处理什么，处理后怎样把学习留给下一轮。

讲者开场用凌晨持续提交的 PR、连续两天运行的 goal loop、每天产出 20–40 个高质量页面来说明价值来源（00:00:00–00:00:21）。后面逐层展开：trigger 唤醒 agent；agent 调查和行动；signal 捕捉业务摩擦和机会；artifact 保存结构化产物；codebase harness 让 agent 能跑、能测、能并行；loop contract 和 global work log 让系统跨会话记住自己做过什么。最终的 support loop 示例把所有部件串成可执行的落地路径。

8.1 讲者的实质性收尾

视频结尾保留了一个实质性建议：讲者创建了名为 `loop engineer setup` 的模板仓库，封装团队搭建 loop 的最佳实践（00:19:48–00:19:55）。读者可以复制这个文件夹，指示 agent 设置 artifacts，并启动新的 loops（00:19:57–00:20:04）。这里给出的实践姿势很具体：先复制一套文件结构和契约模板，再让 agent 结合自己的业务填充 domain loop。

讲者还补充，模板链接放在描述栏，读者可以免费使用；如果想看更完整的过程，还有数小时逐步演示，展示他如何从零真实设置这些循环，并包含十多个小时关于构建 agent、生产环境和 AI coding 的深入讨论（00:20:04–00:20:17）。这些内容应作为资源提示保留在 PDF 的结尾或附录中。常规告别语没有教学信息，可以省略。

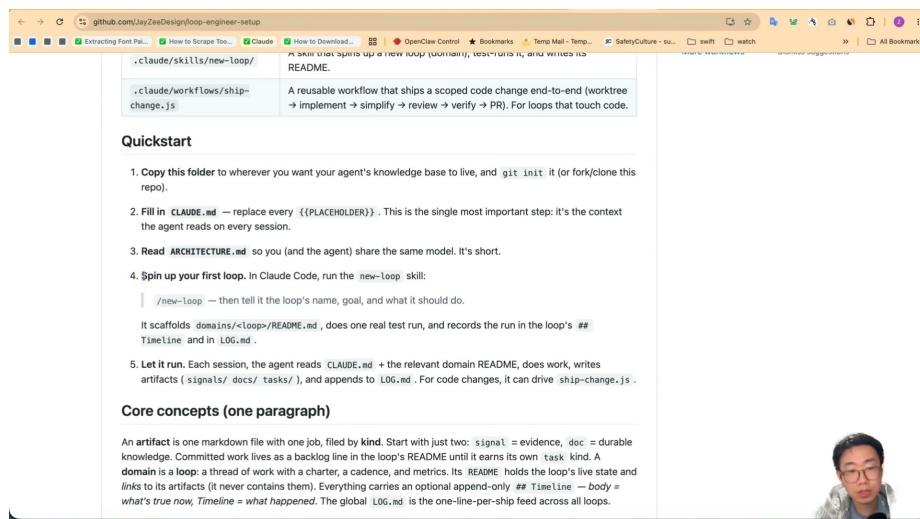


图 16: `loop-engineer-setup` 模板把上下文、架构、artifact、domain loop 和日志规范组织成可复制的 quickstart。¹⁶

¹⁶ 视频画面时间区间：00:19:48–00:20:04。

8.2 一个压缩系统模型

如果把全片压成一句公式，可以写成：

$$\text{valuable loop} = \text{trigger} + \text{state} + \text{artifact} + \text{tools} + \text{verification} + \text{learning}$$

其中：

- **trigger** 决定何时唤醒 loop，可以是 cron、另一个 agent、webhook、故障告警或业务事件。
- **state** 让 agent 知道当前进度，来源包括 loop contract、timeline、global work log 和相关 artifact。
- **artifact** 把 agent 的工作变成可读、可链接、可复查的结构化产物，例如 docs、signals、tasks、campaigns、tickets。
- **tools** 让 agent 接触真实世界，包括 MCP、connectors、skills、浏览器、数据库、支付系统、日志系统和本地脚本。
- **verification** 用测试、浏览器证据、E2E 流程和只读 review agent 降低“自己做自己判卷”的风险。
- **learning** 把本轮观察写回 signal、timeline、contract 和 global work log，使下一轮起点更高。

全片核心判断

Loop Engineering 面向 agent 持续工作的环境，覆盖范围超过单个 prompt。一个有价值的 loop 会在正确时刻被触发，读取正确状态，调用正确工具，产出可维护 artifact，留下验证证据，并把新 signal 写回共享知识层。

这个模型也解释了为什么粗糙 while loop 的价值有限。重复运行同一段 prompt 只能增加尝试次数；引入 state、artifact、verification 和 learning 之后，系统才具备越跑越清楚的可能性。工程上真正困难的地方也在这里：一旦状态、权限、文件结构或验证链路设计不好，agent 的自主性会把错误放大得更快。

8.3 实践落地顺序

讲者把 Loop Engineering 讲成一条可以照着执行的顺序。结合全片内容，推荐的 rollout sequence 是：

1. **让 repo 可读**：建立 AGENTS.md 或类似索引文档，指向架构、业务、脚本、测试和约束文档。agent 首先要知道去哪找信息。
2. **让 repo 可执行**：提供一条命令启动开发环境，配置本地脚本，让 agent 可以在低认知成本下跑起系统。
3. **加入验证路径**：补齐关键 E2E 测试、状态跳转脚本、Playwright 证据、只读 review agent 和 PR 前检查流程。
4. **定义 artifact folders 和 schemas**：为 docs、signals、tasks、tickets、campaigns 等产物写 README、metadata、body、timeline 规则。

5. 定义一个 **domain loop contract**: 先选一个具体 loop, 例如 support loop, 在 domains/support 下写 goal、workflow、boundary、todo、timeline。
6. 手动运行一次: 让 agent 按 contract 处理过去一小时工单, 检查它是否创建了可读 ticket、signal、task 和日志。
7. 通过手动验收后再自动触发: 当人工确认输出质量、权限边界和验证证据都能接受, 再设置每小时或 30 分钟 trigger。

最小可行 loop 的推荐起点

第一个 loop 适合选择支持工单、文档维护、SEO 页面生成或错误日志 triage 这类输入明确、事实源明确、验证相对明确的任务。它们能让团队尽早看见文件结构、contract 和 validation 的真实摩擦。

这条顺序背后的原则是“先人工校准, 再自动持续”。一开始就追求多 loop 并行, 容易把文件格式、权限边界和验证缺口全部隐藏到自动化里; 先跑一个 support loop, 反而能逼出最基础的问题: agent 读不懂业务背景、工具权限不够、ticket schema 不好用、signal 太泛、工程 task 没有验收标准、日志无法帮助下一轮接手。

8.4 风险清单

Loop Engineering 的风险也很现实。以下检查项应该在每个 loop 自动化之前逐项确认:

- **缺少 source-of-truth files**: 业务背景、架构、流程、权限和测试入口没有稳定文件, agent 会从旧上下文或猜测中行动。
- **artifact ownership 不清**: ticket、signal、task、doc 的归属边界含糊, 同一件事会被写到多个地方, 后续 loop 难以判断权威版本。
- **validation evidence 弱**: 只有 agent 自述“完成了”, 没有测试、日志、截图、浏览器录制或只读 review 证据。
- **共享文件夹变成噪音池**: signal 没有来源、重复、严重度和状态, product loop 会被低质量观察淹没。
- **多个 loop 写人不兼容格式**: support、SEO、ads、product growth 各写一套字段, 复利结构会退化成手工整理工作。
- **agent autonomy 超过 human review**: 自动回复客户、修改订阅、创建 PR、触发部署等动作没有分级权限, 事故会跨系统传播。

别把自动化速度误判为系统成熟

loop 跑得快只能说明 trigger 生效。系统是否成熟, 要看它能否留下可靠 artifact、能否引用证据、能否被独立验证、能否在下一轮读取过去的学习并减少重复错误。

风险清单还有一个隐含项: 人类仍然要设计经济边界。support loop 的收益来自减少响应成本和发现产品摩擦; SEO loop 的收益来自稳定内容生产和机会发现; ads loop 的收益来自关键词反馈; product loop 的收益来自更好的优先级。每个 loop 都应有目标指标, 否则自动化会变成“看起来很忙”的后台任务。

8.5 读者应带走的能力

这期视频真正要训练的下一项能力，是设计环境的能力：让 agent 在每次被唤醒时拿到正确工作、正确记忆、正确工具和正确验证路径。Prompt Engineering 仍然有用，它像给单次任务写好指令；Context Engineering 仍然有用，它像把相关资料放进工作台；harness 和 Loop Engineering 则决定这张工作台能否长期支持多个 agent 自主接力。

最终可以把实践判断收束为四句话：

- 没有 trigger，agent 不会在正确时机介入。
- 没有 artifact 和 signal，loop 的学习无法被其他 loop 复用。
- 没有 loop contract 和 global work log，跨会话工作会丢状态。
- 没有 verification，持续自动化会持续放大不可靠输出。

所以，给 agent 更多自由只是表层。Loop Engineering 要求工程师设计一套让自由可追踪、可校准、可验证、可复盘的系统。这个系统一旦跑顺，support、SEO、ads、product growth 等 loop 可以共享同一个 brain，在不同节奏上观察业务、处理任务、沉淀 signal，并把下一轮工作推到更高的起点。

8.6 资源与复盘问题

视频描述中给出的实践资源包括 [loop-engineer-template](#) 仓库，以及 AI Builder Club 的中文页面 [aibuilderclub.com/chinese](#)。这些资源的价值在于提供可复制的文件结构起点：先把 CLAUDE.md、ARCHITECTURE.md、artifact 目录、domain contract 和 LOG.md 放进项目，再让 agent 根据业务场景填充具体 loop。

读者在落地前可以用下面的问题做一次冷启动检查：

- 这个 loop 的 trigger 是时间、事件、告警，还是另一个 agent 的交接？
- agent 每轮需要读取哪些 source-of-truth files，哪些系统才是事实来源？
- 它会产出哪些 artifact，字段、归属边界和 timeline 规则是否清楚？
- 哪些 signal 可以被其他 loop 复用，哪些只是本轮临时观察？
- 哪些动作允许自动执行，哪些动作需要人工确认？
- verification evidence 会以测试结果、浏览器截图、视频、日志，还是只读 review agent 报告的形式留下？
- 如果下一轮 agent 只读取最近 5-10 条日志和 domain contract，它能否知道该继续做什么？

这些问题比“要不要自动化”更具体。真正值得自动化的 loop，应该先能在手动运行中产出清晰 artifact、可信 signal、可复查证据和可接手的下一步。自动 trigger 只是把这个已经被校准过的流程按节奏重复执行。